

17 Complete the function

Now type out of the rest of the function. To show a new shape, we need to change its state from **HIDDEN** to **NORMAL**. The code below does this by using Canvas's `itemconfigure()` function. It uses another Canvas function, `itemcget()`, to update the `current_color` variable, which will be used to check for a snap.

```
previous_color = current_color
```

```
c.delete(shape)
```

Delete the current shape, so that the next shape doesn't show on top of it and so that it won't be shown again.

```
if len(shapes) > 0:
```

Get the next shape if there are any shapes left.

```
    shape = shapes.pop()
```

Make the new shape visible.

```
    c.itemconfigure(shape, state=NORMAL)
```

Assign `current_color` to the colour of the new shape.

```
    current_color = c.itemcget(shape, 'fill')
```

```
    root.after(1000, next_shape)
```

Wait 1 second before showing the next shape.

```
else:
```

```
    c.unbind('q')
```

These lines stop the program responding to snaps after the game is over.

```
    c.unbind('p')
```

```
    if player1_score > player2_score:
```

```
        c.create_text(200, 200, text='Winner: Player 1')
```

```
    elif player2_score > player1_score:
```

```
        c.create_text(200, 200, text='Winner: Player 2')
```

```
    else:
```

```
        c.create_text(200, 200, text='Draw')
```

This code shows the winner on the screen or declares the game a draw.

```
c.pack()
```



EXPERT TIPS

Configuring Canvas items

You can alter things that appear on the canvas by using Canvas's `itemconfigure()` function. In this game, for instance, you use `itemconfigure()` to change shapes from hidden to visible, but you could also use it to change their colour or other characteristics. To use `itemconfigure()`, put the name of the item you want to change in brackets, followed by a comma and then the characteristic and its new value.

